



BATCH AND ROLL NO:
EXPERIMENT NO.9
TITLE: Design a mobile app to store data using internal or external storage.
DATE OF PERFORMANCE
DATE OF CHECKING

Title: Design a mobile app to store data using internal or external storage.

Requirements:

1 Android studio

Theory:

Designing a mobile app to store data using internal or external storage involves creating an application that allows users to store and manage data on their mobile devices. Internal storage refers to the built-in storage on a device, while external storage can include removable SD cards or cloud-based storage solutions. To implement storage functionality in a mobile app, developers typically use platform-specific APIs and libraries, such as Android's SharedPreferences, SQLite database, or File API, or iOS's CoreData or File Manager API. These APIs provide a way to read, write, and manage data stored on the device or in the cloud.

When designing the user interface for a data storage app, developers need to consider how users will interact with the app and manage their data. This can involve creating screens and views that allow users to view, edit, and delete data.

Here are the general steps to design a mobile app to store data using internal or external storage:

1. Define the requirements for your app, including the types of data to be stored, the app's user interface, and any functionality or features that are needed.
2. Choose a development environment, such as Android Studio or Xcode, and create a new project.
3. Determine the appropriate storage method for your app, such as internal storage, external storage, or a cloud-based storage solution.
4. Implement the storage functionality using the appropriate API, such as Android's SharedPreferences, SQLite database, or File API, or iOS's CoreData or File Manager API.
5. Design the user interface for your app, including any screens or views needed to display and manage the stored data.
6. Implement the UI using the appropriate UI components, such as TextViews, EditTexts, and RecyclerViews, and link the UI to the storage functionality using appropriate callbacks or data bindings.
7. Test the app on a physical device or emulator, and make any necessary adjustments to the UI or functionality.



PUNE INSTITUTE OF COMPUTER TECHNOLOGY, PUNE – 411043

Department of Electronics & Telecommunication Engineering

8. Optimize the app's performance and stability, using tools such as Android Profiler or Xcode's Instruments.
9. Add any additional features or functionality as needed, based on user feedback or evolving requirements.
10. Publish the app to the Google Play Store or Apple App Store, following the appropriate guidelines and requirements.



Code:

1. activity_main.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" android:padding="16dp"
    android:layout_marginVertical="150dp"
    android:background="#F0F4F8">
    <TextView android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Data Storage App"
        android:textSize="24sp"
        android:textColor="#2C3E50"
        android:textStyle="bold"
        android:gravity="center"
        android:layout_marginBottom="20dp"/>
    <EditText android:id="@+id/fileNameInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter Filename"
        android:padding="12dp"
        android:background="@drawable/rounded_edittext"
        android:layout_marginBottom="10dp"/>
    <EditText android:id="@+id/dataInput"
        android:layout_width="match_parent"
        android:layout_height="150dp"
        android:hint="Enter Data"
        android:gravity="top"
        android:padding="12dp"
        android:background="@drawable/rounded_edittext"
        android:layout_marginBottom="10dp"/>
    <RadioGroup android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center"
        android:layout_marginBottom="10dp">
        <RadioButton android:id="@+id/internalStorageRadio" android:layout_width="wrap_content"
            android:layout_height="wrap_content" android:text="Internal Storage"
            android:layout_marginEnd="20dp"/>
        <RadioButton android:id="@+id/externalStorageRadio" android:layout_width="wrap_content"
            android:layout_height="wrap_content" android:text="External Storage"/>
    </RadioGroup>
    <LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
        android:orientation="horizontal" android:gravity="center">
        <Button android:layout_width="wrap_content" android:layout_height="wrap_content"
            android:text="Save" android:onClick="saveData" android:layout_marginEnd="10dp"
            style="@style/Widget.MaterialComponents.Button.OutlinedButton"/>
        <Button android:layout_width="wrap_content" android:layout_height="wrap_content"
            android:text="Load" android:onClick="loadData"
            style="@style/Widget.MaterialComponents.Button.OutlinedButton"/>
    </LinearLayout>
</LinearLayout>
```



2. rounded_edittext.xml

```
<shape xmlns:android="http://schemas.android.com/apk/res/android">
    <solid android:color="#FFFFFF"/>
    <corners android:radius="8dp"/>
    <stroke android:width="1dp" android:color="#CCCCCC"/>
</shape>
```

3. Main_Activity.xml

```
package com.example.exp9;
```

```
import android.content.Context;
import android.os.Bundle;
import android.os.Environment;
import android.view.View;
import android.widget.EditText;
import android.widget.RadioButton;
import android.widget.Toast;
```

```
import androidx.appcompat.app.AppCompatActivity;
```

```
import java.io.BufferedReader;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.InputStreamReader;
import java.nio.charset.StandardCharsets;
```

```
public class MainActivity extends AppCompatActivity {
    private EditText dataInput, fileNameInput;
    private RadioButton internalStorageRadio, externalStorageRadio;
```

```
@Override
```

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
```

```
    dataInput = findViewById(R.id.dataInput);
    fileNameInput = findViewById(R.id.fileNameInput);
    internalStorageRadio = findViewById(R.id.internalStorageRadio);
    externalStorageRadio = findViewById(R.id.externalStorageRadio);
}
```

```
public void saveData(View view) {
    String data = dataInput.getText().toString();
    String fileName = fileNameInput.getText().toString();
```

```
    if (data.isEmpty() || fileName.isEmpty()) {
        Toast.makeText(this, "Please enter data and filename", Toast.LENGTH_SHORT).show();
        return;
    }
```

```
    try {
        if (internalStorageRadio.isChecked()) {
            saveInternalStorage(fileName, data);
```



```
} else if (externalStorageRadio.isChecked()) {
    saveExternalStorage(fileName, data);
}
} catch (Exception e) {
    Toast.makeText(this, "Error saving data: " + e.getMessage(),
        Toast.LENGTH_SHORT).show();
}
}

public void loadData(View view) {
    String fileName = fileNameInput.getText().toString();

    if (fileName.isEmpty()) {
        Toast.makeText(this, "Please enter filename", Toast.LENGTH_SHORT).show();
        return;
    }

    try {
        String data;
        if (internalStorageRadio.isChecked()) {
            data = readInternalStorage(fileName);
        } else if (externalStorageRadio.isChecked()) {
            data = readExternalStorage(fileName);
        } else {
            Toast.makeText(this, "Select storage type", Toast.LENGTH_SHORT).show();
            return;
        }
        dataInput.setText(data);
    } catch (Exception e) {
        Toast.makeText(this, "Error reading data: " + e.getMessage(),
            Toast.LENGTH_SHORT).show();
    }
}

private void saveInternalStorage(String fileName, String data) {
    try (FileOutputStream fos = openFileOutput(fileName, Context.MODE_PRIVATE)) {
        fos.write(data.getBytes(StandardCharsets.UTF_8));
        Toast.makeText(this, "Saved to Internal Storage", Toast.LENGTH_SHORT).show();
    } catch (Exception e) {
        Toast.makeText(this, "Internal Storage Error: " + e.getMessage(),
            Toast.LENGTH_SHORT).show();
    }
}

private void saveExternalStorage(String fileName, String data) {
    if (isExternalStorageWritable()) {
        File file = new File(getExternalFilesDir(null), fileName);
        try (FileOutputStream fos = new FileOutputStream(file)) {
            fos.write(data.getBytes(StandardCharsets.UTF_8));
            Toast.makeText(this, "Saved to External Storage", Toast.LENGTH_SHORT).show();
        } catch (Exception e) {
            Toast.makeText(this, "External Storage Error: " + e.getMessage(),
                Toast.LENGTH_SHORT).show();
        }
    } else {
        Toast.makeText(this, "External Storage Not Writable", Toast.LENGTH_SHORT).show();
    }
}
```



```
private String readInternalStorage(String fileName) throws Exception {
    try (FileInputStream fis = openFileInput(fileName);
        InputStreamReader isr = new InputStreamReader(fis, StandardCharsets.UTF_8);
        BufferedReader bufferedReader = new BufferedReader(isr)) {

        StringBuilder sb = new StringBuilder();
        String line;
        while ((line = bufferedReader.readLine()) != null) {
            sb.append(line).append("\n");
        }
        Toast.makeText(this, "Loaded from Internal Storage", Toast.LENGTH_SHORT).show();
        return sb.toString().trim();
    }
}

private String readExternalStorage(String fileName) throws Exception {
    if (isExternalStorageReadable()) {
        File file = new File(getExternalFilesDir(null), fileName);
        try (FileInputStream fis = new FileInputStream(file);
            InputStreamReader isr = new InputStreamReader(fis, StandardCharsets.UTF_8);
            BufferedReader bufferedReader = new BufferedReader(isr)) {

            StringBuilder sb = new StringBuilder();
            String line;
            while ((line = bufferedReader.readLine()) != null) {
                sb.append(line).append("\n");
            }
            Toast.makeText(this, "Loaded from External Storage",
                Toast.LENGTH_SHORT).show();
            return sb.toString().trim();
        }
    } else {
        throw new Exception("External Storage Not Readable");
    }
}

private boolean isExternalStorageWritable() {
    String state = Environment.getExternalStorageState();
    return Environment.MEDIA_MOUNTED.equals(state);
}

private boolean isExternalStorageReadable() {
    String state = Environment.getExternalStorageState();
    return Environment.MEDIA_MOUNTED.equals(state) ||
        Environment.MEDIA_MOUNTED_READ_ONLY.equals(state);
}
```



PUNE INSTITUTE OF COMPUTER TECHNOLOGY, PUNE – 411043
Department of Electronics & Telecommunication Engineering

Output:

EXP_9 Data Storage App Rahul.txt hello ☒ Internal Storage ☐ External Storage SAVE LOAD  Saved to Internal Storage	EXP_9 Data Storage App Rahul.txt hello ☒ Internal Storage ☐ External Storage SAVE LOAD  Loaded from Internal Storage
EXP_9 Data Storage App 42307.txt AD ☐ Internal Storage ☒ External Storage SAVE LOAD  Saved to External Storage	EXP_9 Data Storage App 42307.txt AD ☐ Internal Storage ☒ External Storage SAVE LOAD  Loaded from External Storage



PUNE INSTITUTE OF COMPUTER TECHNOLOGY, PUNE – 411043

Department of Electronics & Telecommunication Engineering

CONCLUSION:

.....

.....

.....